Machine Setup and First Measurements on iccluster113

Extended report — CS-471 project "Tiered Memory Management Beyond Hotness"

Mayeul Besnard Sergio Boffi

November 10, 2026

Abstract

This report describes the server provided for our reproduction of Liu et al. (OSDI'25), what we verified on it, and the first measurements. It extends `Machine_Setup_Report.md` with background explanations and with the work of November 10: the PEBS sampling test, the changes to the authors' microbenchmark, and the first run of experiment E1. All results are **provisional**: they were taken with the CPU frequency governor, turbo boost and automatic NUMA balancing left at their original values. The commands we ran are listed, with their purpose and outcome, in `Commands_Log.md`.

Contents

1 Background **3**
- 1.1 Tiered memory and its emulation . . . 3
- 1.2 Memory-level parallelism and AOL . . . 3
- 1.3 Measuring AOL with hardware counters . . . 3
- 1.4 The slowdown model . . . 4
- 1.5 Counters, `perf` and multiplexing . . . 4
- 1.6 PEBS . . . 4
- 1.7 Linux mechanisms that move memory . . . 4

2 Summary **5**

3 CPU and memory topology **5**

4 Performance counters **6**
- 4.1 Access . . . 6
- 4.2 Events and derived metrics . . . 6
- 4.3 Multiplexing and the two-run procedure . . . 6

5 CPU frequency **7**

6 Kernel tiering state **7**

7 The authors' artifact (SoarAlto) **8**
- 7.1 Structure . . . 8
- 7.2 Findings that matter on our machine . . . 8
- 7.3 SOAR on Haswell . . . 8

8 System settings and tools **9**

9 First measurements with the microbenchmark **9**
9.1 Quick timings 9
9.2 Counters (`-i 9`, 3 runs per configuration) 9
9.3 Pointer chasing without the setup phase 9
9.4 Counter anomaly 10

10 First GAPBS results **10**

11 PEBS sampling test (November 10) **10**

12 Microbenchmark changes for E1 (November 10) **11**

13 E1: first results (November 10, provisional) **12**

14 Original state of the machine **12**

15 Open questions for the TA **13**

16 Next steps **13**

1 Background

This section explains the concepts used in the rest of the report.

1.1 Tiered memory and its emulation

A tiered memory system has a small, fast tier (local DRAM) and a larger, slower tier, such as CXL-attached memory. The operating system decides on which tier each page lives. Systems such as TPP, NBT, Nomad and Colloid move frequently accessed (*hot*) pages to the fast tier.

We have no CXL hardware. Like the authors, we emulate the slow tier with the memory of the *other* socket of a two-socket server. A program runs on the CPUs of node 0, and its memory is placed either on node 0 (fast tier) or on node 1 (slow tier). Every access to node 1 crosses the link between the two sockets, which adds latency and limits bandwidth. The tool `numactl` chooses the CPUs (`--cpunodebind`, `--physcpubind`) and the memory node (`--membind`) of a whole process. The library call `numa_alloc_onnode` places a single buffer.

1.2 Memory-level parallelism and AOL

An out-of-order core does not wait for each cache miss before issuing the next one. If several misses are independent, they are in flight at the same time and their latencies overlap. The average number of misses in flight is the *memory-level parallelism* (MLP).

- In a **pointer chase** each load needs the address returned by the previous one. Misses are serial, MLP is about 1, and every access pays the whole memory latency.
- In a **sequential read** the addresses are known in advance. Many misses overlap and the hardware prefetchers fetch lines early. MLP is high, and a slower memory costs much less than its latency suggests.

The paper's metric is the **Amortized Offcore Latency**, AOL = Latency/MLP: the latency each miss *effectively* costs once overlap is taken into account. The paper's main claim is that the slowdown caused by slow memory depends on AOL, not on how often data is accessed. A frequently accessed array with high MLP can tolerate the slow tier better than a rarely accessed linked list.

1.3 Measuring AOL with hardware counters

Intel CPUs count off-core requests, i.e. requests that leave the core towards L3 and memory. Three counters are needed:

Event	Meaning
`offcore_requests.demand_data_rd` (N)	number of demand data reads
`offcore_requests_outstanding.demand_data_rd` (O)	every cycle, adds the number of reads in flight
`offcore_requests_outstanding.cycles_with_demand_data_rd` (C)	cycles with at least one read in flight

O is the sum, over all requests, of their latencies (Little's law). Therefore

$$\text{Latency} = \frac{O}{N}, \qquad \text{MLP} = \frac{O}{C}, \qquad \text{AOL} = \frac{\text{Latency}}{\text{MLP}} = \frac{C}{N}.$$

AOL does not depend on O. This turned out to be important, because O is unreliable on our machine (Section 9.4).

1.4 The slowdown model

The paper models the slowdown on the slow tier as $S = P \times K(\text{AOL})$:

- P is the fraction of cycles in which the core is stalled waiting for memory. A program that rarely waits for memory cannot slow down much.
- $K(\text{AOL}) = 1/(a + b/\text{AOL})$ grows with AOL: misses that are not hidden by parallelism make the program more sensitive. The authors' scripts use $a = 0.87$ and $b = 24.67$, fitted on Skylake. We must fit them again on our machine (experiment E2).

On Skylake P is measured with `cycle_activity.stalls_l3_miss`. This event does not exist on our CPU (Section 4).

1.5 Counters, `perf` and multiplexing

`perf stat` counts events for one command. A CPU has a few fixed counters and, on our CPU, 4 programmable counters per hardware thread when hyper-threading is on. If more events are requested than counters are free, the kernel *multiplexes*: it rotates the events on the counters and scales the counts. `perf` then prints a percentage such as `(61.27%)`, the fraction of time the event was actually counted. Scaled values are estimates, so we avoid multiplexing altogether.

1.6 PEBS

Precise Event-Based Sampling makes the CPU write a record every n occurrences of an event, e.g. every 3001 loads that miss L3. For memory events the record contains the address of the instruction and the address of the *data*. SOAR uses these samples to find out which allocated objects cause the misses. The record format depends on the CPU generation; from memory, a timestamp field was added only with format 3 (Skylake).

1.7 Linux mechanisms that move memory

- **NUMA balancing** (`kernel.numa_balancing` = 1) periodically samples the pages of a process and migrates them towards the node that accesses them. It can undo a placement chosen on purpose.
- **Memory tiering mode** (= 2, used by NBT) promotes hot pages from a slower tier to a faster one, and **demotion** moves cold pages down. Both work only between *different memory tiers*, which the kernel builds from each node's "abstract distance".
- **CPU frequency**: with a load-dependent governor and turbo, the clock changes during a run, so the same latency in nanoseconds corresponds to a different number of cycles.

2 Summary

Item	Result	Status
CPU	Intel Xeon E5-2680 v3 (Haswell-EP), 2 sockets × 12 cores, HT on (48 logical CPUs)	older than the paper's Skylake
Memory	2 NUMA nodes, about 126 GB each, almost all free	OK
Kernel	`6.8.0-147-generic` (Ubuntu)	OK
`perf`	v6.8.12, usable with `sudo`	OK
AOL counters	all available	OK
P counter	`stalls_l3_miss` missing; `stalls_l2_pending` as proxy	methodology change
Counter budget	4 programmable counters ⇒ two runs per measurement	solved
Multiplexing	none in any result file (checked on Nov. 9)	OK
Frequency	`intel_cpufreq`, `schedutil`, turbo on	to be fixed (TA)
NUMA balancing	on; only 209 pages migrated during E1	to be turned off (TA)
Memory tiers	a single tier with both nodes	NBT/TPP cannot migrate
PEBS	format 2, no timestamp; `perf record -d` works	SOAR profiling feasible
Uncore frequency	driver not available	natural remote latency
Slow tier	pointer chasing ≈102 ns local, ≈154 ns remote (1.5×)	less slow than the paper (2.1×)
Counter anomaly	O unreliable; AOL $= C/N$ consistent	AOL derived without O
GAPBS	`bc`, `bfs`, `pr`, `cc`: +29% to +37% on the remote node	provisional
Microbenchmark	three changes for E1, verified	done
E1	first run done; whole-program time still missing	provisional
SoarAlto artifact	full SOAR pipeline, but Skylake-specific; baselines are kernel patches	porting needed

3 CPU and memory topology

```
$ lscpu | grep -E "Model name|Vendor|Socket|Core\(s\)|Thread\(s\)|NUMA|^CPU\(s\)|L3"
CPU(s):                  48
Model name:              Intel(R) Xeon(R) CPU E5-2680 v3 @ 2.50GHz
Thread(s) per core:      2
Core(s) per socket:      12
Socket(s):               2
L3 cache:                60 MiB (2 instances)
NUMA node0 CPU(s):       0,2,4,...,46   (even)
NUMA node1 CPU(s):       1,3,5,...,47   (odd)

$ numactl -H
node 0 size: 128686 MB   free: 127073 MB
node 1 size: 129012 MB   free: 128267 MB
node distances:   0: 10 21   1: 21 10
```

- **12 cores per socket** are more than the 8 threads used in the paper.
- **30 MiB of L3 per socket** is far smaller than the workloads (several GB), so they really exercise memory.

- **About 126 GB per node**: any workload fits entirely on one node, which is what static placement experiments need.
- **CPU numbering is interleaved**: even CPUs are on node 0, odd CPUs on node 1. A range such as `0-11` would spread threads over both sockets. This is exactly the bug we found in the microbenchmark (Section 12).
- **Hyper-thread siblings**: CPU i and CPU $i + 24$ share a physical core. CPUs `0,2,4,6,8,10,12,14` are 8 distinct physical cores of node 0; we use them for multi-threaded runs.
- The NUMA distance 21 is an estimate written by the firmware, not a measurement. The real latencies are in Section 9.

4 Performance counters

4.1 Access

`/proc/sys/kernel/perf_event_paranoid` is `4`, a Debian/Ubuntu value that forbids counters to normal users. We run `sudo perf` for each measurement, instead of lowering this setting, so the machine configuration stays unchanged.

4.2 Events and derived metrics

All AOL events exist. For P we use `cycle_activity.stalls_l2_pending`, the cycles stalled while an L2 miss is pending.

Quantity	Formula
Latency (cycles)	O/N
MLP	O/C
AOL (cycles)	C/N
P	`stalls_l2_pending` / `cycles`

Differences from the paper, due to Haswell:

- `stalls_l2_pending` also counts stalls on L2 misses that hit in L3. Our P therefore includes stalls the slow tier does not affect, and is probably overestimated.
- There are no offcore events restricted to L3 misses. Latency and AOL mix L3 hits (about 40 cycles) with DRAM accesses, so AOL can be "diluted" for cache-friendly workloads.
- A possible correction, to be evaluated in E2, uses Intel's Top-Down method for Haswell. From memory, and still to be checked against Intel's documentation:

$$P_{\text{DRAM}} \approx \left(1 - \frac{h}{h + 7m}\right) \times \frac{\text{stalls_l2_pending}}{\text{cycles}},$$

with h = `mem_load_uops_retired.l3_hit` and m = `mem_load_uops_retired.l3_miss`. It would need a third run per measurement.

4.3 Multiplexing and the two-run procedure

A first test with six events showed multiplexing on `cycles` (61%) and `stalls_l2_pending` (39%). The cause is the NMI watchdog (`nmi_watchdog` = 1): it uses the fixed cycle counter, so our `cycles` must take a programmable one, and with hyper-threading only four are available. Five programmable events do not fit.

Instead of disabling the watchdog or hyper-threading, which would change the machine, each measurement is split into two runs of the same command, both verified to run without multiplexing:

```
# Run A - AOL
sudo perf stat -e cycles,instructions,offcore_requests.demand_data_rd,\
offcore_requests_outstanding.demand_data_rd,\
offcore_requests_outstanding.cycles_with_demand_data_rd -- <cmd>
# Run B - P
sudo perf stat -e cycles,instructions,cycle_activity.stalls_l2_pending -- <cmd>
```

The price is that P and AOL come from different executions; with stable runs (standard deviation below 1%) this is acceptable. On November 9 we checked all result files for multiplexing percentages: there are none.

5 CPU frequency

```
scaling_driver: intel_cpufreq      scaling_governor: schedutil      no_turbo: 0
cpuinfo_min/max_freq: 1200000 / 3300000 kHz
available governors: conservative ondemand userspace powersave performance schedutil
```

`intel_cpufreq` is `intel_pstate` in passive mode, so the generic governors can be used. With `schedutil` and turbo, the frequency moves between 1.2 and 3.3 GHz depending on load and temperature. For final measurements we plan, after TA approval, the `performance` governor and turbo off (fixed 2.5 GHz), restoring the original values afterwards.

6 Kernel tiering state

```
$ uname -r; cat /proc/sys/kernel/numa_balancing; cat /sys/kernel/mm/numa/demotion_enabled
6.8.0-147-generic
1
false
$ cat /sys/devices/virtual/memory_tiering/memory_tier*/nodelist
0-1                        (a single tier: memory_tier4)
```

- **NUMA balancing is on.** It could move our pages during a run. During E1 the system-wide counter `numa_pages_migrated` grew by only 209 pages, against about 500 000 pages in our buffers, so it did not matter there. It should still be off for final runs.
- **Both nodes are in the same memory tier.** Promotion and demotion only move pages between tiers, so NBT and TPP would never migrate anything. Taking node 1's CPUs offline, as the authors do, is probably not enough, since the tier depends on the abstract distance set by the memory driver (to be verified). The artifact contains a module, `run/colloid/tpp/tierinit/tierinit.c`, that moves node 1 to a lower tier, but it relies on helpers defined in the Colloid kernel patch.

Experiment	Works on the stock kernel?
E1 (static placements)	yes (`numactl`, `numa_alloc_onnode`)
E2 (counters, calibration of K)	yes
SOAR profiling	yes, with `perf record -d` (Section 11)
SOAR placement (`LD_PRELOAD` interception)	yes by code review; not tested
E3 baselines NBT, TPP	no: two memory tiers needed
E3 baselines Nomad, Colloid	no: custom kernels

7 The authors' artifact (SoarAlto)

7.1 Structure

Path	Content
`src/microbenchmark/`	pointer-chasing and sequential microbenchmark (E1)
`src/soar/`	SOAR: allocation tracing (`prof/ldlib.c`), sampling (`run/prof.sh`), object scoring (`run/proc_obj_e.py` → `obj_stat.csv`), placement by interception (`interc/ldlib.c`, policy in `check_trace`), patches (`nbt.patch`, `gapbs.patch`)
`src/alto/`	kernel patches and build scripts for TPP, NBT, Nomad, Colloid, with and without ALTO
`run/`	`setup.sh`, `config.sh`, and `bc-urand/` (the only workload with a run directory)

7.2 Findings that matter on our machine

1. **SOAR does not need the authors' kernel.** The SOAR README asks for Linux 5.18 with `nbt.patch`. By inspection, the only part of the patch SOAR uses is a `pebs:pebs` tracepoint (instruction address, data address, timestamp); the rest concerns NBT migrations. Our PEBS format has no timestamp, so the tracepoint would be useless anyway, and `perf record -d` replaces it (Section 11).
2. **Slow-tier emulation.** The authors lower the uncore frequency of the remote socket to 500 MHz (about 190 ns latency) through `/sys/devices/system/cpu/intel_uncore_frequency`. This driver is not available on our machine. The alternative is writing MSR `0x620`, which needs TA approval; otherwise we keep the natural remote latency.
3. **Local DRAM capacity is limited with the GRUB `memmap=` parameter**, a persistent boot change that conflicts with the TA rule that the stock kernel stays the default. Reserving 1 GB huge pages on node 0 is a possible alternative (untested).
4. **Baselines** are built by scripts that switch kernel version, patch, install and update the bootloader. They must be adapted to the one-time GRUB boot procedure. "Colloid" in the artifact is Colloid-TPP, with a Skylake-specific patch.
5. **Scripts must be read before running.** `src/soar/run/prof.sh` calls `check_cxl_conf`, which disables the NMI watchdog, ASLR, KSM, NUMA balancing, THP, hyper-threading, turbo and swap and takes node 1's CPUs offline. It must not be run as is.

7.3 SOAR on Haswell

Phase	Work required
Allocation tracing (`prof/ldlib.c`)	change the hard-coded output path; may need a fix for recent `g++` (`throw(std::bad_alloc)`)
Sampling (`run/prof.sh`)	replace the tracepoint with `perf record -d` (done in principle); align the `perf` clock with the `rdtsc` timestamps of `ldlib.c`; fix hard-coded paths
Object scoring (`proc_obj_e.py`)	Haswell event names; recalibrated a, b and AOL thresholds (40, 45, 60, 80)
Placement (`interc/ldlib.c`)	replace the call-site addresses in `check_trace` with ours
Workloads	GAPBS built with `-no-pie` (done); Intel MLC not available

8 System settings and tools

```
THP: always [madvise] never      ASLR: 2      KSM: 0      SMT: on
GRUB_DEFAULT=0   GRUB_TIMEOUT_STYLE=hidden   GRUB_TIMEOUT=0   empty command lines
1 GB huge pages on node 0: 0
df: 162 GB free; /, /boot and /home on the same partition
```

- **THP** is `madvise`: plain `malloc` does not get huge pages. Our pointer chase therefore also suffers TLB misses with 4 KB pages.
- **ASLR** is on. With `-no-pie`, the call-site addresses SOAR relies on should stay fixed, so ASLR may not need to be disabled.
- **GRUB**: `GRUB_DEFAULT=0` boots the first menu entry, and the menu is hidden with no timeout. A newly installed custom kernel with a higher version could become the default, and could not be avoided by hand at boot. This must be fixed before any custom kernel.
- **Tools**: `gcc`, `make` and `libnuma-dev` were missing. We installed `build-essential` and `libnuma-dev` with `apt`.

9 First measurements with the microbenchmark

Conditions: unmodified microbenchmark, one thread on node 0, the whole process's memory on node 0 or node 1 (`numactl --membind`), 1 GiB buffer.

9.1 Quick timings

Pattern (`-i 3`)	Node 0 (s)	Node 1 (s)	Slowdown
Pointer chasing	15.2	22.0	+45%
Sequential read	8.9	11.6	+30%

The low-MLP pattern slows down more, as the paper predicts. These times include the construction and shuffling of the list.

9.2 Counters (`-i 9`, 3 runs per configuration)

Configuration	Time (s)	Latency	MLP	AOL	P	Slowdown
Pointer chasing, node 0	25.49	603	3.7	162	0.76	—
Pointer chasing, node 1	37.01	988	4.2	236	0.79	+45%
Sequential, node 0	23.46	183	11.7	16	0.66	—
Sequential, node 1	31.42	252	12.3	21	0.74	+34%

Latency and AOL are in cycles. P is similar for the two patterns, while AOL differs by a factor of 10, which is the kind of difference the model needs. Latency and MLP in this table are not reliable (Section 9.4).

9.3 Pointer chasing without the setup phase

The setup phase is the same for any number of iterations. Subtracting a run with `-i 1` from the run with `-i 9` leaves 8 traversals, about 134 million steps:

Quantity (`-i 9` minus `-i 1`)	Node 0	Node 1
Time per step	**≈102 ns**	**≈154 ns**
Core cycles per step	≈253	≈383
C per step	≈254	≈385
Requests per step	1.35	1.38

The remote node is about **1.5**× slower, against about 2.1× (90 vs 190 ns) in the paper. The time per step includes TLB misses, so it is not a pure memory latency. Our slow tier is therefore milder than the paper's, and this must be stated in the methodology.

9.4 Counter anomaly

A pointer chase is serial, so its MLP should be close to 1. We measure 4.2–5.2, also after removing the setup phase, and a latency of 791 cycles while one step takes 253 cycles: a request cannot be outstanding for longer than the step that waits for it. Hence O **(`offcore_requests_outstanding.demand_data_rd`) is not reliable on this machine**, at least for this pattern. The cause is unknown (a Haswell erratum or an effect of hyper-threading are possible).

C matches the real cycles per step, so **AOL** $= C/N$ **remains usable**. The authors' `proc_obj_e.py` computes it the same way. Latency and MLP derived from O will not be reported for pointer chasing.

10 First GAPBS results

Conditions: GAPBS built with `-no-pie`; graph `urand24` (16.8 M vertices, 2.2 GB) read once into the page cache from node 0; 8 OpenMP threads on cores `0,2,...,14`; memory on node 0 or node 1; 3 runs with each event set. Arguments: `bc -i4 -n1`, `pr -n1`, `bfs -n8`, `cc -n8`. `sssp` needs a weighted graph and `tc` is very slow, so they were not run. Summary: `gapbs_summary.log` in the repository.

Workload	Local (s)	Remote (s)	Slowdown	Whole process	P (loc./rem.)	AOL (loc./rem.)
`bc`	17.32	23.75	**+37.1%**	+35.2%	0.78 / 0.84	46 / 63
`bfs`	0.24	0.33	**+35.4%**	+25.2%	0.56 / 0.64	35 / 48
`pr`	4.29	5.74	**+33.8%**	+30.1%	0.87 / 0.90	28 / 38
`cc`	0.59	0.76	**+29.1%**	+23.6%	0.65 / 0.72	62 / 82

The slowdown uses GAPBS's *Average Time*, which covers the algorithm only.

- The slowdowns are close to each other (29–37%), partly because our slow tier is only 1.5× slower. A narrow range leaves little for a predictor to explain.
- The correlation between local P and slowdown is weak (about 0.2), but four points say nothing about the hypotheses yet.
- The counters cover the whole process, including graph loading. For `bfs` and `cc` the algorithm is a small part of the run, so their P and AOL mostly describe loading. More trials (`-n`) are needed.

11 PEBS sampling test (November 10)

Question: can we collect what SOAR needs (L3-miss samples with data address and time) on the stock kernel?

```
sudo perf record -d -e mem_load_uops_retired.l3_miss:P -c 3001 \
     -o ~/hotness/results/pebs_test.data -- \
     numactl --cpunodebind=0 --membind=0 ./bench -R 0.0 -i 1 -A 1024
```

`:P` requests precise sampling (PEBS), `-c 3001` takes one sample every 3001 events as the authors do, and `-d` records the data address.

Result: 17 814 samples. Where they fall:

Function	Samples
`shuffle` (list setup)	33.4%
`pc_thread` (traversal; `ptr_chase` is inlined here)	31.4%
`init_ptr_buf` (list setup)	31.1%
kernel	≈4%

Samples inside `pc_thread` look like this (time in seconds, data address, instruction address, function):

```
1894399.201677:     749a8bdbfc80     5c6c278f00e6 pc_thread
1894399.201993:     749a88f4f640     5c6c278f00e6 pc_thread
1894399.202305:     749a89de5080     5c6c278f00e6 pc_thread
```

Three independent checks show that the samples are correct:

1. The data addresses differ at every sample and are multiples of 64: they are list nodes, one cache line each. The instruction address is always the same load of the loop.
2. The addresses lie within about 57 MB, consistent with the list being shuffled in blocks of about 64 MB.
3. Samples are about 0.316 ms apart, i.e. 3001 misses, or **≈105 ns per miss**: the same as the 102 ns per step measured from execution times.

Conclusions:

- **SOAR profiling is feasible on this machine**, on the stock kernel, without the authors' patch or Linux 5.18.
- The timestamps come from the kernel's `perf` clock (time since boot), not from the PEBS record. The remaining work is to align them with the `rdtsc` timestamps written by `ldlib.c`, e.g. by switching `ldlib.c` to the same clock or by selecting a clock with the `-k` option of `perf record` (to be tested).
- With one iteration, two thirds of the misses come from the setup phase. This explains why whole-program counters were dominated by setup.

12 Microbenchmark changes for E1 (November 10)

E1 runs two threads at the same time: one chases pointers in buffer A (a 1 GiB linked list, low MLP), the other reads buffer B sequentially (a 1 GiB array, high MLP). The array is the *hot* data: with `-i 9` it is read $9 \times 26 = 234$ times, about 3.9 billion cache-line accesses, against about 151 million for the list. The unmodified program could not run this experiment. We made three changes on branch `haswell-port` of our clone of SoarAlto.

1. **Pinning on one socket.** Thread i was pinned to CPU i, so the two threads ran on CPUs 0 and 1, which are on different sockets here. Now thread i is pinned to CPU $2i$. Check: the two worker threads run on CPUs 0 and 2, both on node 0.
2. **One memory node per buffer.** Both buffers came from plain `malloc`; the existing function `init_buf` (which calls `numa_alloc_onnode`) was never used, and option `-r` had no effect. Both threads now use `init_buf` and free with `numa_free`; `-r` selects the node of buffer A, and a new option `-b` selects the node of buffer B. Buffer B is allocated 4096 bytes larger, because `bandwidth()` writes 8 bytes past its end, which could crash once the buffer ends exactly at a page boundary. Check with `numastat`: about 1025 MB on node 0 and 1024 MB on node 1 with `-r 0 -b 1`.
3. **Per-thread timing.** The program printed nothing. Each thread now measures only its work loop with `clock_gettime(CLOCK_MONOTONIC)` and prints `pc_time_s` or `bw_time_s`. Check: with `-i 1`, `pc_time_s` is 1.72 s, i.e. 102 ns × 16.8 M steps, as expected.

13 E1: first results (November 10, provisional)

Four placements, 5 repetitions each, `-i 9`, both buffers 1 GiB. The names refer to the hotness-based view, in which the array is the hot data.

Configuration	List / array node	Pointer chasing time (s)	Pointer chasing slowdown	Sequential time (s)	Sequential slowdown
`all_dram`	0 / 0	15.74	—	22.62	—
`all_slow`	1 / 1	24.16	+53.5%	30.94	+36.8%
`hot_on_dram`	1 / 0	23.28	+47.8%	22.69	+0.3%
`cold_on_dram`	0 / 1	16.06	+2.0%	30.85	+36.4%

The placement stayed static: `numa_pages_migrated` grew by 209 pages in the whole system during the 20 runs.

Reading the results:

- **In relative terms, `cold_on_dram` wins.** Placing the cold but latency-bound list in DRAM costs the two threads about 38% in total, against 48% for the hotness-based choice. This is the direction of H1.
- **The sequential read suffers more than expected (+36%).** It reads about 11 GB/s from node 0 and 8 GB/s from node 1 (234 GiB in 22.6 and 30.9 s). On this machine the remote array seems limited by the bandwidth of the link between the sockets, not only by latency, so its MLP protects it less than in the paper.
- **In seconds lost, the two placements are close:** about 7.6 s for `hot_on_dram` and 8.5 s for `cold_on_dram`, because the array thread does far more work.
- **The decisive metric is missing:** the time of the whole program. The program ends with the slower thread, and the list thread also spends about 10 s in its setup, which accesses memory too. A run with whole-program time (3 repetitions) is the next step.

14 Original state of the machine

To be restored after any experiment that changes it.

Setting	Original value
scaling_governor	schedutil
intel_pstate/no_turbo	0
scaling_min_freq / scaling_max_freq	1200000 / 3300000
kernel.numa_balancing	1
numa/demotion_enabled	false
kernel.nmi_watchdog	1
kernel.perf_event_paranoid	4
transparent_hugepage/enabled	madvise
kernel.randomize_va_space	2
ksm/run	0
smt/control	on
1 GB huge pages on node 0	0
GRUB_DEFAULT / GRUB_TIMEOUT / GRUB_TIMEOUT_STYLE	0 / 0 / hidden
Kernel command line additions	none
Kernel	6.8.0-147-generic

Changes made so far: packages `build-essential` and `libnuma-dev`; files under `~/hotness/`; a repository-local git identity in our clone of SoarAlto. Possibly also the `msr` module and `msr-tools`, suggested by another chat; whether they were run is unconfirmed (see `Commands_Log.md`). No system setting has been changed.

15 Open questions for the TA

1. During measurements, may we set the governor to `performance`, disable turbo and set `numa_balancing` to 0, restoring the values right afterwards?
2. Both NUMA nodes are in one memory tier, so NBT and TPP cannot migrate. Is there a recommended way to make node 1 a lower tier?
3. The artifact limits local DRAM with the GRUB `memmap=` parameter. Is that allowed (e.g. for a one-time boot only), or should we use another method, such as 1 GB huge pages?
4. The uncore frequency driver is not available. May we set the remote uncore frequency through MSR `0x620`, or should we keep the natural remote latency (1.5×)?
5. Which of the artifact's settings (HT, swap, ASLR, KSM, THP, NMI watchdog, node 1's CPUs offline) may we apply during our runs?
6. We installed `build-essential` and `libnuma-dev`. Is that fine, and may we also install `vmtouch` and `libgfortran5`?
7. Could we get CloudLab access (c220g5, the authors' Skylake platform) for the baselines?
8. Who else uses this machine, and is a remote console available if a kernel does not boot?

16 Next steps

1. E1 with whole-program time; copy `e1.log` to the repository and plot.
2. Repeat E1 and the counter measurements under stable settings, after TA approval.
3. E2: more varied workloads (more GAPBS trials, a pointer chase with N independent chains to vary MLP in a controlled way), then fit a and b.
4. SOAR: align the `perf` and `ldlib.c` clocks, port the scoring script, then try one workload.

5. Record the commit hashes of SoarAlto and GAPBS.
6. Before any custom kernel: persistent journal, `/boot` space check, one-time GRUB boot.